

Trading engine prompt — conversation log

Iterative refinement of a Claude Code prompt for building a backtesting engine

Kushagra

improve the below prompt to get the most out of claude code for building a trading and backtesting engine.

Role:

You are an expert in making trading systems and backtesting.

Objective

On a new .py file build a trading engine basis the following rules:

General

source dataframe to build on: features_d

Initial account size = 100_000_000

Max allocation to one security = 50% of portfolio value at any time. Keep this 50% as tunable parameter

Take the day's close as the price for buying or selling a security

Buy condition:

if there are no units of a security, features_d['buy'] == 1

If there are > 0 units of a security, check the following conditions:

features_d['buy'] == 1 AND

(current buy price > previous buy price OR current buy price > ema21)

Hold condition:

If a security reaches 50% portfolio allocation, stop further buying. Keep holding till stop loss is hit

If a security does not reach 50% allocation but the cash values becomes 0 or less than that required to

Sell Condition:

If the stop loss is hit, sell all units at once.

Trade Log:

During each Buy or sell event, log:

1. Trade count - counted as 1 for the respective buy and sell events. If stop loss is hit then trade count

1. buy / sell price

2. timestamps

3. weighted average buy price (weighted by number of units)

4. Gains absolute and in percentage when liquidating a security

Position Sizing

1. Max risk when the price is at the lower half of the base is 1%. that is for an account size of 100_000_000

2. Judge the stop loss and max risk at every new buy trade

Trade Summary and Evaluation

Create a summary of total trades, trades by security, portfolio cum gains, annualized gains, sharpe ratio

Create functions to plot and view the same over the whole time horizon

Create any other feature that you may feel is important to log

Sort the features_d by date so that it is chronologically sorted before running the trades.

Store data in a format (tabular) which can be easily accessed to create a dashboard that allows a user

On the other hand it should also show any stock that is showing a sell signal. The sell signal stock should

On another tab of the dashboard we should be able to see the performance of the portfolio till date going

Do not create the dashboard just yet, but have the data ready so that the dashboard can be created seamlessly

Claude

Here is a significantly improved version, tightened for Claude Code's strengths — explicit schemas, unambiguous logic, modular file structure, and clear output contracts.

[An architecture diagram was rendered inline in the chat showing: features_d input → engine.py (main loop) drawing from config.py, with position_manager.py, trade_logger.py, and analytics.py as sub-modules; analytics writes Parquet files for downstream dashboard use.]

Here is the improved prompt. Copy it directly into Claude Code:

```
# Trading & Backtesting Engine – Build Instructions for Claude Code

## Preamble for Claude Code
Before writing any code:
1. Read this entire spec carefully.
2. Identify any remaining ambiguities and resolve them by making explicit, documented assumptions in a `ASSUMPTIONS.md` file at the project root. Do not silently guess.
3. Build the project as a set of clean, importable Python modules – not a single script.
4. After writing each module, run a syntax check (`python -m py_compile <file>`) and fix any errors before moving on.
5. At the end, run a smoke test using a minimal synthetic `features_d` DataFrame to verify the full pipeline executes without errors.

---

## Project structure

Create the following files:

trading_engine/
■■■■ ASSUMPTIONS.md          # Documented assumptions you made during build
■■■■ config.py               # All tunable parameters as named constants
■■■■ engine.py               # Main backtest loop – entry point
■■■■ position_manager.py     # Position sizing, allocation limits, stop-loss logic
■■■■ trade_logger.py         # Trade book and event logging
■■■■ analytics.py            # Performance metrics, plots, output serialization
■■■■ smoke_test.py           # Minimal end-to-end test with synthetic data

---

## Input data contract

The backtest consumes a single DataFrame called `features_d`. It must have these columns:

| Column          | Dtype       | Description                                                                                               |
|-----|-----|-----|
| date            | datetime64  | Trading date (one row per security per date) |
| ticker          | str         | Security identifier |
| close           | float       | Day's closing price – used as both buy and sell execution price |
| buy             | int (0/1)   | Buy signal for this security on this date |
| ema21           | float       | 21-period EMA of close |
| stop_loss       | float       | Absolute stop-loss price level for this security on this date |

**Sort first**: At the start of `engine.py`, sort `features_d` by `['date', 'ticker']` ascending before any iteration. Never assume the input is pre-sorted.

**Validate on load**: Assert that all required columns are present and that `stop_loss` < `close` for every row where `buy == 1`. Log a warning (do not raise) for any row that violates this.

---

## config.py – tunable parameters

INITIAL_CAPITAL      = 100_000_000  # Starting portfolio cash (INR or USD, unitless)
MAX_SECURITY_ALLOC   = 0.50         # Max fraction of portfolio value in one security
MAX_RISK_PER_TRADE   = 0.01         # Max capital at risk per new buy (1% of portfolio)
EXECUTION_PRICE_COL  = "close"      # Which price column to use for fills

All logic that references these values must import them from `config.py`.
```

They must never be hardcoded elsewhere.

engine.py – backtest loop

State you must track per security (per-ticker dict or a small dataclass):

```
units_held          : int      # Current units owned
avg_buy_price       : float    # Weighted average buy price (by units)
last_buy_price      : float    # Price of the most recent buy for this ticker
allocated_value     : float    # Current market value held (units × current close)
trade_count         : int      # Increments with each buy or sell event
```

Portfolio-level state:

```
cash                : float    # Available cash
portfolio_value     : float    # cash + sum of all open positions at current close
```

Recompute `portfolio\_value` at the start of each new date before processing any signals.

Buy logic (evaluate in this exact order)

Case 1 – No existing position in this ticker:

Trigger: `features\_d['buy'] == 1` AND `units\_held == 0`

Case 2 – Existing position (add-on buy):

Trigger: ALL of the following:

- `features\_d['buy'] == 1`
- `units\_held > 0`
- `current\_close > last\_buy\_price` OR `current\_close > ema21`

Allocation gate (applies to both cases):

Before executing any buy:

1. Compute $\text{current_alloc} = (\text{units_held} \times \text{current_close}) / \text{portfolio_value}$
2. If $\text{current_alloc} \geq \text{MAX_SECURITY_ALLOC}$ → skip buy, log "max allocation reached"
3. Compute position_size (see Position Sizing below)
4. If $\text{cash} < \text{current_close}$ (cannot afford even 1 unit) → skip buy, log "insufficient cash"
5. Cap position size so that total allocation after buy does not exceed $\text{MAX_SECURITY_ALLOC} \times \text{portfolio_value}$

Position sizing (called on every new buy):

```
stop_loss_pct = (current_close - stop_loss_price) / current_close
               # stop_loss_price comes from features_d['stop_loss']
```

```
position_size = (MAX_RISK_PER_TRADE × portfolio_value) / stop_loss_pct
               # e.g. (0.01 × 100_000_000) / 0.15 = 6_666_666.67
```

If $\text{stop_loss_pct} \leq 0$ or $\text{stop_loss_price} \geq \text{current_close}$, skip the buy and log a warning – do not divide by zero or negative values.

```
units_to_buy = floor(min(position_size, available_headroom) / current_close)
```

where $\text{available_headroom} = \text{MAX_SECURITY_ALLOC} \times \text{portfolio_value} - \text{units_held} \times \text{current_close}$

If $\text{units_to_buy} == 0$, skip the buy.

After a buy executes:

- Deduct $\text{units_to_buy} \times \text{current_close}$ from cash
- Update units_held , avg_buy_price (weighted), last_buy_price
- Increment trade_count
- Log the trade event (see Trade Log)

Sell logic

Trigger: `current_close <= stop_loss_price` (stop-loss hit)

Sell ALL `units_held` at `current_close` in a single transaction.

- Add `units_held × current_close` to cash
- Record realized gain/loss
- Reset all per-ticker state to zero
- Increment `trade_count`
- Log the trade event
- Note: `trade_count` for the NEXT buy in this ticker continues from current value + 1

`trade_logger.py` – event log schema

Append one row per event (buy or sell). Store as a list of dicts, convertible to DataFrame:

<code>event_id</code>	: int	# Auto-incrementing global event ID
<code>trade_count</code>	: int	# Trade count for this ticker
<code>ticker</code>	: str	
<code>event_type</code>	: str	# "BUY" or "SELL"
<code>date</code>	: date	
<code>execution_price</code>	: float	
<code>units_transacted</code>	: int	
<code>units_held_after</code>	: int	
<code>avg_buy_price</code>	: float	# Weighted avg; NaN after full sell
<code>stop_loss_price</code>	: float	# stop_loss at time of this event
<code>stop_loss_pct</code>	: float	# $(\text{exec_price} - \text{stop_loss}) / \text{exec_price}$
<code>position_size_used</code>	: float	# Capital deployed in this transaction
<code>portfolio_value</code>	: float	# Portfolio value at time of event
<code>cash_after</code>	: float	
<code>realized_gain_abs</code>	: float	# Only populated on SELL; else NaN
<code>realized_gain_pct</code>	: float	# $(\text{sell_proceeds} - \text{cost_basis}) / \text{cost_basis}$; else NaN

Dashboard data outputs (write at end of backtest)

Serialize all DataFrames to **Parquet** files in an `outputs/`` directory.
Also write CSV equivalents for manual inspection.

File 1: `outputs/trade_log.parquet`
The full event log from `trade_logger.py`.

File 2: `outputs/daily_signals.parquet`
One row per (date, ticker). Include columns:

<code>date</code> , <code>ticker</code> , <code>close</code> , <code>stop_loss</code> , <code>stop_loss_pct</code> ,	
<code>buy_signal</code> ,	# 1 or 0
<code>sell_signal</code> ,	# 1 if stop_loss hit and ticker is in portfolio, else 0
<code>in_portfolio</code> ,	# bool – does the engine hold units at start of this date?
<code>units_held</code> ,	
<code>avg_buy_price</code> ,	
<code>position_size_estimate</code> ,	# what <code>position_size()</code> would return if a buy were triggered
<code>units_purchaseable</code> ,	# $\text{floor}(\text{position_size_estimate} / \text{close})$
<code>portfolio_value_bod</code> ,	# portfolio value at start of day (before today's trades)

This is the primary feed for the dashboard's "buy candidates" and "sell signals" views.

File 3: `outputs/portfolio_daily.parquet`
One row per date (portfolio-level):

<code>date</code> ,	
<code>portfolio_value</code> ,	
<code>cash</code> ,	
<code>invested_value</code> ,	# <code>portfolio_value - cash</code>
<code>daily_return</code> ,	# pct change from prior day

```

cumulative_return,      # from day 0

### File 4: outputs/trade_summary.parquet
One row per closed trade (buy → sell pair):

ticker,
trade_count,
buy_date, sell_date,
avg_buy_price, sell_price,
units, cost_basis, proceeds,
realized_gain_abs, realized_gain_pct,
holding_days

---

## analytics.py – metrics and plots

Implement the following functions. Each plot function must save the figure to outputs/plots/
as a PNG and also return the matplotlib Figure object.

def compute_summary_stats(portfolio_daily, trade_summary) -> dict:
    """
    Returns a dict with:
    - total_trades (count of completed buy→sell pairs)
    - win_rate (fraction of trades with realized_gain_pct > 0)
    - avg_gain_pct, median_gain_pct
    - total_realized_gain_abs
    - portfolio_cumulative_return_pct
    - annualized_return_pct (CAGR: (end/start)^(365/calendar_days) - 1)
    - sharpe_ratio (annualized; use 252 trading days; risk-free = 0)
    - max_drawdown_pct
    - calmar_ratio (annualized_return / abs(max_drawdown))
    - avg_holding_days
    """

def plot_portfolio_curve(portfolio_daily) -> plt.Figure:
    """Cumulative return curve over time."""

def plot_drawdown(portfolio_daily) -> plt.Figure:
    """Rolling drawdown from peak over time."""

def plot_trade_distribution(trade_summary) -> plt.Figure:
    """Histogram of realized_gain_pct across all closed trades."""

def plot_by_ticker(trade_summary) -> plt.Figure:
    """Bar chart: total realized gain per ticker, sorted descending."""

---

## Coding standards

- Explicit arguments only – no mutable default arguments, no global state outside config.py.
- Type hints on all function signatures.
- Docstrings on all public functions (one-line minimum).
- Use logging (not print) for all runtime messages. Set level to INFO by default.
- No pandas chained assignment – use .loc[] or .assign().
- Handle ZeroDivisionError and KeyError defensively in the main loop; log and continue.
- smoke_test.py must be runnable as `python smoke_test.py` and exit with code 0 on success.

```

The key gaps I filled in vs your original draft:

Stop-loss was never defined. Your original said "if stop loss is hit" but never said what `stop_loss` was or where it came from. The improved version makes `features_d['stop_loss']` an explicit required column with a validation step.

Position sizing formula was ambiguous. "Lower half of the base" is not computable from the DataFrame you described. I've simplified it to a single clean formula — $(\text{MAX_RISK_PER_TRADE} \times \text{portfolio_value}) / \text{stop_loss_pct}$ — and left a note in `ASSUMPTIONS.md` for you to fill in if the base-half logic needs to vary the risk percentage.

Multiple buy gates were unordered. The add-on buy has three conditions and an allocation cap. The improved version gives Claude Code a strict evaluation order so it can't mis-sequence them.

Dashboard data was underspecified. "Store in tabular format" is too vague. The improved version names four Parquet files with exact column lists so Claude Code can build the analytics layer independently from the UI layer later.

Coding standards were absent. Claude Code will write much cleaner, debug-friendly code when you give it explicit constraints on logging, type hints, and argument style — especially since this codebase will eventually connect to a live dashboard.

Kushagra

I already have a Claude Code session running. Should I paste this prompt there or start a new Claude Code session in PyCharm?

Claude

Start a new session. Here's why:

The prompt is a greenfield spec — it defines its own file structure, data contracts, and assumptions from scratch. An existing session carries accumulated context (prior file edits, earlier instructions, intermediate decisions) that can subtly pull Claude Code away from the clean architecture the prompt lays out. You'd essentially be asking it to hold two mental models at once.

A few practical things to do before you paste:

Navigate to the right directory first. In your terminal inside PyCharm, `cd` into wherever you want `trading_engine/` to live within your `markets` project before launching the new Claude Code session. Claude Code anchors its file operations to the working directory it starts in.

One addition to make before pasting. Add a short context block at the very top of the prompt so Claude Code knows the environment without having to ask:

```
## Environment context
- Python environment: conda env named `markets`
- Project root: <your actual path>
- Key dependency already installed: pandas, numpy, matplotlib, pyarrow
- Do NOT use .env files for any config; all parameters go in config.py
- Target Python: 3.10+
```

Then paste the full prompt as your very first message in the new session, not broken into pieces. Claude Code does better when it sees the complete spec upfront rather than receiving it incrementally — it can plan the module structure before writing a single line.

Your existing session stays open in PyCharm as a separate tab, so you don't lose any prior context there if you need to reference it.

Kushagra

I have created a lot of code pertaining to data acquisition and engineering. Hope it will not alter those files or functions.

Claude

Valid concern. A few layers of protection to put in place:

The most important thing — tell Claude Code explicitly in the prompt. Add this block right after the Environment context section:

```
## Boundaries – do NOT touch these
This project already has existing code for data acquisition and feature engineering.
You must ONLY create files inside the `trading_engine/` directory.
Do not read, edit, import from, or reference any existing `.py` files outside
that directory unless I explicitly ask you to.
If you need sample data to run the smoke test, generate a synthetic DataFrame
inline in `smoke_test.py` – do not import from any existing pipeline.
```

Two practical safeguards on your end before you start:

- Commit your current state to Git right now** before starting the new session. That way even if something goes sideways, a `git diff` will show you exactly what changed and `git checkout` brings it back instantly.
- Glance at your PyCharm `.idea/` run configurations** — Claude Code occasionally picks up existing module names from the project tree. If your existing files have names that overlap with the new ones (`config.py` is a common collision), rename one side before starting.

One thing to know about Claude Code's behavior: it is generally good about staying within its working directory, but it *will* sometimes read nearby files for context if they're in scope. Starting the session from inside the `trading_engine/` subdirectory rather than the project root gives you a tighter blast radius — it simply won't see files outside that folder unless you explicitly reference them.

The Git commit is the real safety net though. Do that first.

Kushagra

I am okay for it to refer to the files created as the `features_d` dataframe will be used to create this. and this dataframe is created as part of the feature engineering process.

Claude

Good call — that actually makes the build cleaner. Update that boundary block to this instead:

```
## Boundaries
- You MAY read existing project files to understand how `features_d` is constructed
  - its exact column names, dtypes, and index structure.
- You must NOT edit, refactor, or add to any existing file outside `trading_engine/`.
- Treat all existing files as read-only reference material only.
- Once you have read the feature engineering code, document the confirmed `features_d`
  schema in ASSUMPTIONS.md before writing any engine logic.
```

And add this as the **first instruction** in the prompt, before anything else:

```
## First step – schema discovery
Before writing any code, read the existing feature engineering files and identify:
1. The exact column names present in `features_d`
2. Their dtypes
3. Whether `stop_loss` exists as a column – if not, flag this immediately and
   propose how it should be derived (e.g. from a base low price column)
4. Whether `ema21` exists or is named differently
```

Document your findings in ASSUMPTIONS.md, then confirm with me before proceeding.

That last point about `stop_loss` is important — your VCP pipeline likely computes a base low or pivot level somewhere, but it may not be explicitly named `stop_loss` in `features_d`. Better Claude Code surfaces that gap before building the whole engine around a column name that doesn't exist, rather than failing silently at runtime.

Kushagra

I would also like to create an optuna workflow where i can tune some tunable parameters for maximizing CAGR and sharpe ratio. Would it make sense to pass that as a separate prompt once the trading engine is built, or would you suggest i pass it here only?

Claude

Pass it as a **separate prompt** after the engine is built and verified.

Here is the reasoning:

Building order matters. Optuna needs to call the engine's backtest function in a tight loop — potentially thousands of times. That means the engine needs to be built first as a clean, callable function with a well-defined return signature before Optuna can wrap around it. If you specify both together, Claude Code may over-engineer the engine structure to accommodate tuning before the core logic is even correct.

Verification gap. You want to manually run the engine on real `features_d` data at least once and sanity-check the trade log before handing it to an optimizer. Optuna tuning garbage logic just finds the best parameters for garbage — you won't catch logical errors in the buy/sell conditions until you've eyeballed a few actual trades.

The interface you'll need to define between the two is simple. When you're ready for the Optuna prompt, the engine just needs one clean entry point:

```
def run_backtest(features_d: pd.DataFrame, params: dict) -> dict:
    """
    Returns at minimum:
    - cagr: float
    - sharpe: float
    - max_drawdown: float
    """
```

If Claude Code builds this as the engine's callable interface from the start (which the current prompt already encourages via `compute_summary_stats`), the Optuna layer slots in cleanly later.

What to pass in the Optuna prompt when ready. Keep a note of these for later:

```
Parameters to tune:
- MAX_SECURITY_ALLOC      (e.g. search space: 0.20 to 0.50)
- MAX_RISK_PER_TRADE      (e.g. 0.005 to 0.02)

Objectives:
- Maximize Sharpe ratio (primary)
- Maximize CAGR (secondary)
- Use multi-objective: optuna.create_study(directions=["maximize", "maximize"])

Constraints:
- Discard any trial where max_drawdown > 35%
- Minimum 10 completed trades for a trial to be valid

Practical:
- n_trials = 100 to start
- Use TimeSeriesSplit or walk-forward folds if data allows
- Persist study with optuna's RDB storage so trials survive session restarts
```

One thing to flag now though — add this one line to the current engine prompt so Claude Code builds with Optuna in mind structurally, without actually implementing it yet:

```
## Note for future extensibility
The `run_backtest()` function in engine.py must accept a `params: dict` argument
that overrides config.py defaults. This is required for hyperparameter tuning
downstream. Do not implement tuning now – just ensure the interface supports it.
```

That single line costs nothing now and saves a refactor later.